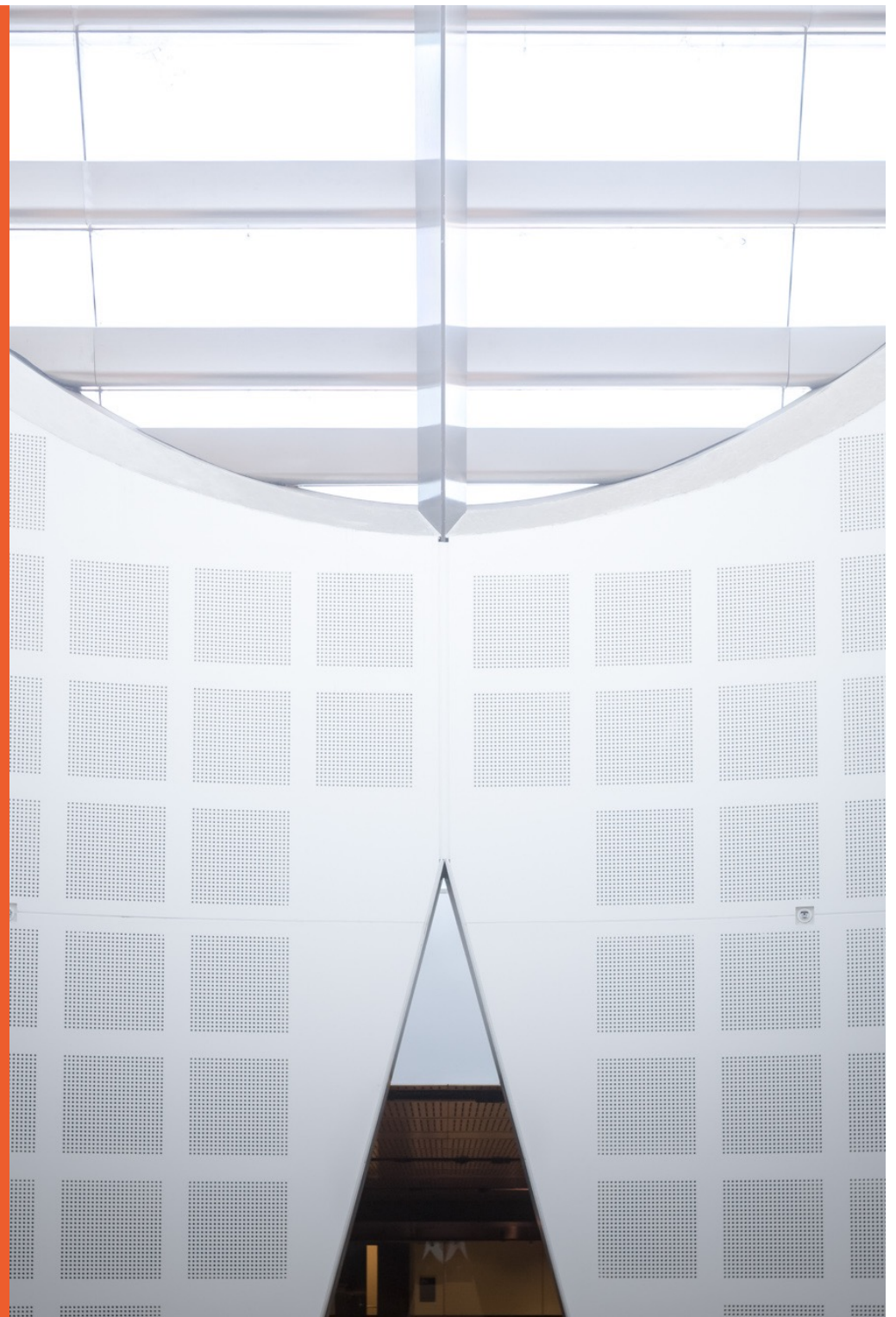


Web Security

Presented by
Dr. Huaming Chen

ELEC3609/9609
Internet Software Platforms
2025 S2



We acknowledge the tradition of
custodianship and law of the Country on which
the University of Sydney campuses stand.
We pay our respects to those who have cared
and continue to care for Country.



THE UNIVERSITY OF
SYDNEY

Overview

✓ Getting Hacked

- ✓ Assessing Threats
- ✓ Web Security Resources

✓ Web Vulnerabilities

- ✓ SQLi
- ✓ XSS
- ✓ CSRF
- ✓ Auth & Sessions
 - ✓ Hashing Passwords
 - ✓ Password Salting
 - ✓ Authentication Tokens
- ✓ LFI/RFI
- ✓ RCE
- ✓ DDoS

✓ Case Studies

Getting Hacked

Getting Hacked: How?

How do sites get hacked?

Because system designers:

- Protect the **wrong things**.
- Protect the **right things** in the **wrong way**.
- Make **poor assumptions** about their systems.

Getting Hacked: How?

How do sites get hacked?

Because system designers:

- Protect the **wrong things**.
- Protect the **right things** in the **wrong way**.
- Make **poor assumptions** about their systems.
- Fail to understand the **threat model**.
- Fail to account for **paradigm shifts** (e.g. the Internet).
- Fail to understand the **scope** of their system.



McDonald's

7m on screen –

1h 21m background

1h 28m

McDonald's collects personal information in connection with providing the MyMcDonald's Rewards program to consumers. This includes the following categories of personal information:

Stealing passwords from McDonald's users

Posted on 6 January 2017 by Tijme Gommers.

By abusing an insecure **cryptographic storage vulnerability** and a **reflected server cross-site-scripting vulnerability** it is possible to steal and decrypt the password from a McDonald's user. Besides that, other personal details like the user's name, address & contact details can be stolen too.

Forbes

Billionaires

Innovation

Leadership

Money

Business

Small Business

Lifestyle

116,228 views | Aug 31, 2019, 03:41am

Critical 'Backdoor Attack' Warning Issued For 60 Million WordPress Users



Davey Winder Senior Contributor @

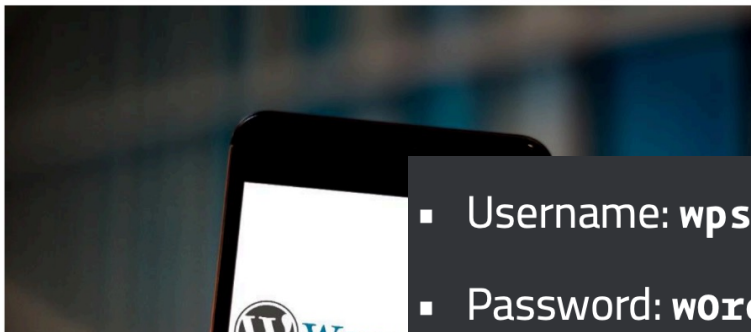
Cybersecurity

I report and analyse breaking cybersecurity and privacy stories

f

t

in



- Username: **wp**services
- Password: **w**ordpr3ss

Identifiers, such as participants' name, email address, (optional) day and month of birth, and MyMcDonald's Rewards account numbers;

Personal information described in Cal. Civil Code

§ 1798.80(e), such as phone number and payment information (such as payment card details);

- **Commercial information**, such as records of members' purchases, information about actions taken while using the McDonald's app, and members' rewards account credentials;
- **Internet or other electronic network information** reflecting a participants' interaction with the McDonald's app, in-restaurant technologies, or ads in connection with the MyMcDonald's Rewards program;
- **Geolocation data**, which may be collected if participants enable their devices to collect their location (and which can be disabled at any time in device settings); and
- **Inferences** drawn from any of the above categories of information to create a profile about a consumer reflecting the consumer's preferences, characteristics, psychological trends, predispositions, behavior, attitudes, intelligence, abilities, or aptitudes.

Web security is hard!

Modern web applications are built on a tangle of technologies that have been developed over time and then haphazardly pieced together. Every piece of the web application stack, from HTTP requests to browser-side scripts, comes with important yet subtle security consequences. To keep users safe, it is essential for developers to confidently navigate this landscape.

-- Michal Zalewski, from Tangled Web

Getting Hacked: What?

What does it mean to get hacked?

Servers:

- The most obvious way to attack a website is to compromise the server(s) running the website. This is done through manipulation of the server-side code:
 - SQL Injection
 - Remote Code Execution
 - Local/Remote File Inclusion

Users:

- A less obvious method of hacking is to attack the users, whilst they use the web server for its intended purpose. This is done through altering or intercepting data on a user's computer:
 - Cross-Site Scripting (XSS)
 - Cross-Site Request Forgery (CSRF)

What Can be Hacked?

A product or component...

- e.g., software program, cryptographic protocol, smart card

...plus, infrastructure

- e.g., PC, operating system, communications

...plus, applications

- e.g., web server, payroll system

...plus, IT staff

...plus, users and management

...plus, customers and external users

...plus, partners and vendors

It's a lot easier to break a system than to make it secure.

BIZ & IT —

How a college student tricked 17k coders into running his sketchy script

Infecting military and government software engineers is easier than you may think.

DAN GOODIN - 6/15/2016, 12:10 AM



ShahamB

78

A German university student has demonstrated an effective way to get code of his choosing to run on the computers of software developers, at least some of whom work for US governmental and military organizations.

The eye-opening (if ethically questionable) research was conducted by University of Hamburg student Nikolai Philipp Tschacher as part of his bachelor thesis. Using a variation of a decade-old attack known as typosquatting, he uploaded his code to three popular developer communities and gave them names that were similar to widely used packages already submitted by other users. Over a span of several months, his imposter code was executed more than 45,000 times on more than 17,000 separate domains, and more than half the time his code was given all-powerful administrative rights. Two of the affected domains ended in .mil, an indication that people inside the US military had run his script.

Axioms of Information Security

- Information security is a resource game.
- All systems are buggy.
- The bigger the system, the buggier it is.
- Nothing works in isolation.
- Humans are most often the weakest link a system.
- It's a lot easier to break a system than to make it secure.

Focus on the Important Risks

1. **Severe:** Must be managed by senior management with a detailed plan.
2. **High:** Detailed research and management required at senior levels.
3. **Major:** Senior management attention is needed.
4. **Significant:** Management responsibility must be specified.
5. **Moderate:** Managed by specific monitoring or response procedures.
6. **Low:** Managed by routine procedures.
7. **Trivial:** Unlikely to need specific application of resources.

Need to find a balance between:

- **Likelihood** of the component being exploited.
- **Damage** if exploited.
- **Mitigation cost** to prevent exploitation.

Resources on Security

- The [Open Web Application Security Project](#) (OWASP) aims to provide comprehensive assessment and education regarding web vulnerabilities.
- The [Common Vulnerabilities and Exposures](#) (CVE) system for publicly known breaches and data leaks. There are over 160,000 tracked incidents.
- The [Damn Vulnerable Web Application](#) (DVWA) is a very simple web application which demonstrates basic web vulnerabilities.
- [PentesterLab](#) includes practical tutorials and exercises to learn various forms of hacking and penetration testing skills, some of which involve exploitation of web vulnerabilities.

Other Courses on Security - ECE

ELEC5616: Computer & Network Security

– Semester 1

Ask for Permission

- The only context within which use of these techniques is allowed, in the eyes of the law, is when it has been explicitly permitted by the owners and administrators of the networks or applications you are testing.
- A sound knowledge of these tools and techniques open doors to plenty of opportunities to utilise them in a commercial setting.
- This may form a very rewarding career – don't throw it away doing something stupid.

Legal vs. Illegal Hacking

The difference is:

Permission

Ask for Permission – AWS/Cloud

- Cloud providers can prohibit certain security tools from being used, particularly those that scan network port ranges or send malicious/malformed traffic.
- Usually this can be waived by submitting a request to scan ahead of time.

Web Vulnerabilities

Web Vulnerabilities - Overview

The OWASP top 10 project identifies the most widespread vulnerabilities on the internet. We will discuss some of these, but there are many more.

- SQLi: SQL Injection
- XSS: Cross-Site Scripting
- CSRF: Cross-Site Request Forgery
- Auth: Authentication & Sessions
- Config: Secure Configurations
- LFI/RFI: Local/Remote File Inclusion
- RCE: Remote Code Execution
- DDoS: Distributed Denial of Service

SQL Injection

- SQL Injection arises from failing to properly sanitize user input before inserting it into a database string.
- If the input can be used to modify an SQL query, then it is possible to modify it to do any database query.
- Very common.
- Can lead to complete compromise.
- Many automated tools to detect (also many to automatically exploit).
- Using an ORM or database library can defend against this.

SQL Injection: Example

```
// Get input
$id = $_REQUEST[ 'id' ];

// Check database
$query = "SELECT first_name , last_name FROM users
WHERE user_id = '$id'";
$result = mysql_query($query)

// Get results
$row = mysqli_fetch_assoc( $result );

// Get values
$first = $row["first_name"];
$last = $row["last_name"];

// Feedback for end user
$html .= "<p>First name: {$first} Surname:
{$last}</p>";
```

SQL Injection: Example

If we access the website with:

<http://example.com/?id=15>

the code will respond normally, adding the details for a user with id 15.

However, if we access the website with:

<http://example.com/?id=%27%3B+DROP+TABLE+users%3B+-->

then the query that gets executed will be:

```
SELECT first_name, last_name FROM users WHERE user_id =  
    '' ; DROP TABLE users; --';
```

SQL Comments

The -- characters indicate the rest of the query is a comment and should be ignored.

SQL Injection: sqlmap

Performing an SQL injection has now been automated. A program called `sqlmap` automates most of the hard work involved with SQLi, and is fully open source (written in Python).

```
$ python sqlmap.py -u "http://debiandev/sqlmap/mysql/get_int.php?id=1" --batch
{1.0.5.63#dev}
http://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is
illegal. It is the end user's responsibility to obey all applicable local, state and fed
eral laws. Developers assume no liability and are not responsible for any misuse or damage
caused by this program

[*] starting at 17:43:06

[17:43:06] [INFO] testing connection to the target URL
[17:43:06] [INFO] heuristics detected web page charset 'ascii'
[17:43:06] [INFO] testing if the target URL is stable
[17:43:07] [INFO] target URL is stable
[17:43:07] [INFO] testing if GET parameter 'id' is dynamic
[17:43:07] [INFO] confirming that GET parameter 'id' is dynamic
[17:43:07] [INFO] GET parameter 'id' is dynamic
[17:43:07] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable
(possible DBMS: 'MySQL')
```

sqlmap demos

Check out the demos on the homepage: sqlmap.org

Cross Site Scripting (XSS)

Cross Site Scripting arises when an attacker is able to **inject HTML** into the website.

Remember, HTML tells the browser how to display a webpage, and can incorporate JavaScript.

Cross Site Scripting (XSS)

- For example, we had a HTML template like this to insert the user's name into the heading of the page:

```
<H1>Hello {% user.name %}</H1>
```

- This works fine for regular users, but what if we had a user whose name was:

```
<script>alert("LOL");</script>
```

- In this case, loading the page will render the following HTML snippet, which will display an alert when this user loads the page:

```
<H1>Hello <script>alert("LOL");</script></H1>
```

- In this example, an alert is only shown to this particular user, but if this snippet were running on a public page (e.g., a list of users, or a blog post showing an author's name) and was more malicious than a simple `alert`, the effects can be devastating.

XSS: Stealing Cookies

‘...the effects can be devastating.’

What’s the problem with a hacker being able to execute JavaScript in your browser?

The answer: **they can steal your cookies!**

If the injected JavaScript was something like:

```
document.write('')`
```

Then the hacker’s server will receive the cookie for any user who loads a page containing the snippet. Now that the hacker has a user’s cookie, they can authenticate as that user.

Preventing XSS

How can we stop XSS?

- One approach is to just strip all `<script>` tags from user input.
 - It's not immediately obvious why, but this is WRONG.
`<s<script>cript>...</script>`
would be turned into:
`<script>...</script>`
- There are many, many ways to perform XSS attacks, and it's impossible to recognise them all.
 - For example, you can use an `<img>` tag:
``

XSS Filter Evasion

Take a look at OWASP's cheat sheet for XSS filter evasion:

www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet

Preventing XSS

The best way to stop XSS is to do so via HTML escaping with our templating library.

1. User input is stored in the database as-is, even if it includes HTML tags.
2. Upon displaying any data, it is HTML escaped by default.

In the case from before, we had a user with the name:

```
<script>alert("LOL");</script>
```

When HTML escaped, the output will be:

```
<H1>Hello<script>alert("&quot;LOL&quot;");  
;</script></H1>
```

This will print the HTML to *the screen* – it will not be interpreted as actual HTML and rendered by the browser.

HTML Characters

< and > are ways to specify < and > respectively.

More info: www.w3schools.com/html/html_symbols.asp

Cross Site Request Forgery (CSRF)

- Cross Site Request Forgery (CSRF) is a type of exploit against a website.
- CSRF exploits arise when only the browser is authenticated (e.g., via a cookie), but the user's request is not properly authenticated.

CSRF Example

Imagine YouTube allowed you to like a video by visiting a link like this:

`www.youtube.com/like?v=Qmr23196`

If I want many likes on my video, I just need to convince others to visit this link while they are logged in (authenticated by *cookies*) as the browser will send along the other user's authentication credentials.

We can make a hidden image on any webpage:

```

```

Or we can make it a link:

```
<a href="www.y.com/like?v=Qmr23196" >Free Smileys!!! </a>
```

Everyone who attempts to view my web page or click my misleading link ends up liking my video.

Preventing CSRF

Typically, we prevent CSRF by including a CSRF token when making requests which change data (e.g. POST, but not GET).

When submitting requests to the server, the website can use JavaScript to also submit a special token from a cookie as one of the POST request variables, which only it has access to.

This CSRF token typically appears somewhere in the HTML or in the cookies.

Checking for the presence of a CSRF token in the cookie is not enough

- it must be included as a variable to authenticate the request.

Authentication

We can consider authentication to occur in two stages:

1. Logging in

- e.g., username and password

2. Identifying logged in session

- e.g., cookies

Authentication

Logging in is pretty straightforward on most websites:

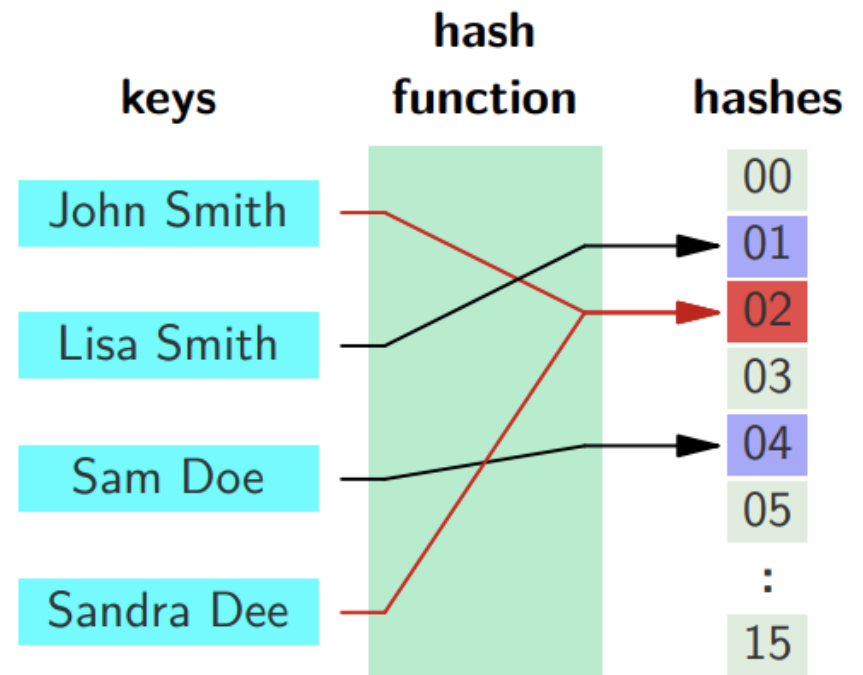
1. User sees login form and provides username and password.
2. Server hashes the password and compares the computed hash with its stored password hash for the user.
3. If the hashes match, the user has been authenticated and the server responds with an authentication cookie/token.

Password Storage

- If we stored passwords in our database as regular plaintext, an attacker who gains access to the database could authenticate as anyone.
- As such, we store password hashes, so that if the database is breached, an attacker would not be able to use the information as-is.
- To use the hashes, an attacker would need to reverse the hash to get the original passwords.

Hash Functions: Overview

- Hash functions are functions that map an input string to an output hash.
- Desirable properties:
 - One way
 - Given a hash, the original string cannot be computed.
 - Unpredictable
 - Cannot choose an input string to get a particular output hash.
 - Non-collision
 - Cannot find two input strings that map to the same output hash.



Cracking Passwords

- If an attacker wanted to reverse some hashes obtained from our DB to discover the original passwords, they could theoretically compute the hash of all possible passwords and match it up to their hash list.
- This is possible because passwords are hard to make up and remember – most human-generated passwords can be found in a dictionary, either as a word or a concatenation of multiple words.
- There is a major extension to this attack: rainbow tables. These contain lists of commonly used passwords (e.g., password1, ilikecats123, etc.) and are used to supplement dictionary attacks with additional words.
- These attacks won't work on every user (e.g., if the password is unique gibberish), but it's been shown to work on large portions of leaked password hash sets.

Salting Passwords

To mitigate the effects of a **dictionary attack** without forcing stricter password requirements on users, passwords can instead be salted.

Before being fed into the hash function, the raw password is concatenated with another string, which is known as the salt.

- Salts are usually stored in plaintext in the database, right next to the password hashes.
 - For further security, a different salt can be used for each user.
- This thwarts dictionary attacks that rely on massive precomputation.
- Once the salt is known however, brute-force dictionary attacks may be run.

User	Salt	Hash
userA	saltA	$h(\text{passwordA} \parallel \text{saltA})$
userB	saltB	$h(\text{passwordB} \parallel \text{saltB})$

Table: An example database table that handles password hashing and salting itself.

Authentication Cookies

- After a user has provided their password and been authenticated, they are provided with an auth token, which is provided in the cookies.
- There are two main ways to manage this token:
 - **Statefully**: store valid tokens & expiry in a database (e.g., Redis).
 - **Statelessly**: calculate valid tokens using a hash of details and a secret.
- Storing valid tokens in a database is obvious, but calculated tokens are less so.

Calculated Authentication Tokens

- Calculated authentication tokens allow the server to check if a user has been authenticated without storing tokens.
- This is done by calculating a value which can be re-computed after every request, then checking if the provided token matches the calculated one.

Authentication Token

```
hash(secret_a || validity_timestamp || user_id || secret_b)
```

- While this adds computational overhead (since the token must be re-computed for every request), it reduces the memory overhead.

Calculated Authentication Tokens

```
def build_auth_token(self):
    max_validity_duration_seconds = 21600 # 6 hours
    timestamp = int(
        datetime.datetime.utcnow().strftime("%s")
    )
    validity_timestamp = int(
        timestamp - (timestamp %
        max_validity_duration_seconds)
    )
    auth_token = hashlib.sha512(
        self.secret_key[0] +
        str(validity_timestamp) +
        str(self.id) +
        self.secret_key[1]
    )
    return auth_token.hexdigest()

def validate_auth_token(self, token):
    return self.build_auth_token() == token;
```

Storing/Sending Cookies

Some tips for storing and managing cookies:

- Always access the website over HTTPS.
 - This prevents network MITM attacks from sniffing auth cookies.
- Set the cookie's secure attribute.
 - This ensures the cookie will only be sent if connecting via HTTPS.
- Set the cookie's HttpOnly attribute.
 - This ensures that scripts on the page cannot read the cookie (e.g., via XSS), and that cookies are only sent with HTTP(S) requests.

Local/Remote File Inclusion

- Local/Remote File Inclusion (file:///) is a vulnerability where the web application is manipulated into loading an unexpected file. This can be used to obtain the contents of a file or dynamically change the behaviour of the program.

```
$page = $_GET["page"];  
include($page);
```

- Simply visiting the following site:
<http://example.com/index.php?page=/etc/passwd>
- will output the `/etc/passwd` file if not configured correctly.
- While this is a very simple example and looks quite obvious, this is a very common bug.
- **Local** = files already on the server, as above.
- **Remote** = get the server to load a file from a remote location.
- **Don't allow user parameters to govern what files can be included.**

Remote Code Execution

- Remote/Arbitrary Code Execution refers to a vulnerability where an attack can provide some code which will be run on the remote server.
- This is considered the worst type of exploit, as it quickly leads to full server compromise.
- There are a few ways this can happen, but the most common culprits are:
 - The `eval()` function (or equivalent).
 - PHP `unserialize()`.
 - Regular Expressions with the 'e' flag.

RCE: Example

- Here is a simple example of an RCE vulnerability, taken from the [OWASP Code Injection](#) page:

```
$myvar = "varname";  
$x = $_GET['arg'];  
eval("\$myvar = \$x;");
```

```
/index.php?arg=1; system('id')
```

```
/index.php?arg=1; phpinfo()
```

Distributed Denial of Service

- Denial of Service attacks prevent anyone from accessing a particular website.
- Typically, this is performed by flooding the website with a very large amount of traffic so that legitimate user requests cannot be processed.
- Defence is very difficult – detection mechanisms can introduce a large amount of overhead for legitimate requests. The only real way to defend your system is to ‘absorb’ the attack traffic.



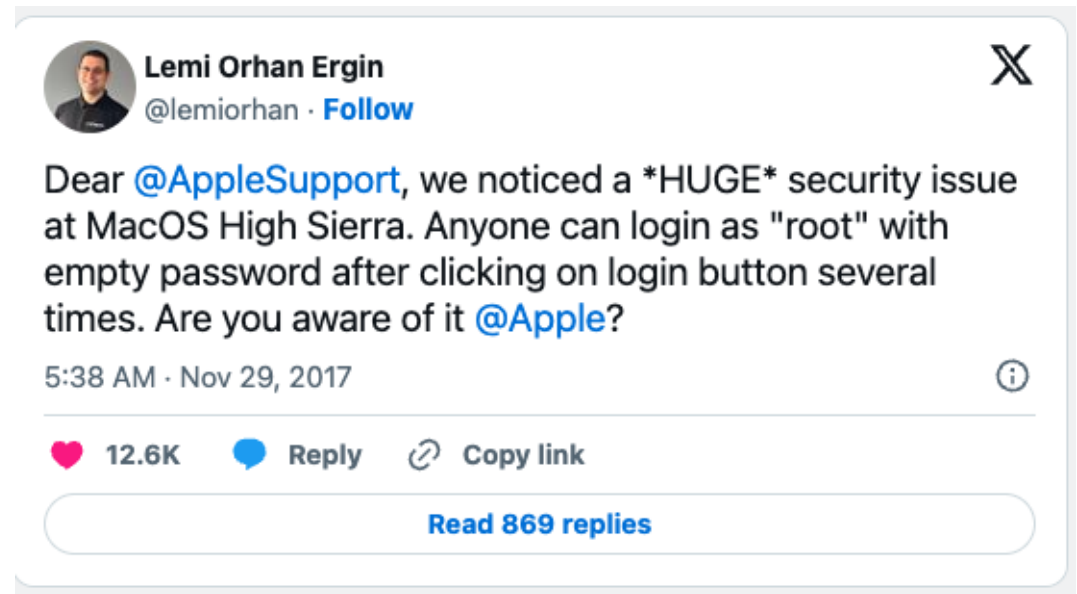
Case Studies

Empty Password

- How many websites/services/operating systems have allowed login when an empty password is submitted for a user?

Empty Password

- How many websites/services/operating systems have allowed login when an empty password is submitted for a user?
 - Apple/macOS High Sierra (2017): macOS bug lets you log in as admin with no password required [Article](#)
 - Valve/Steam (2015): [Article](#)
 - Linux-PAM < 1.5.1: [CVE-2020-27780](#)



Default Account with Empty Password

- How many websites/services/operating systems have default accounts with empty passwords, allowing anyone to login?

Default Account with Empty Password

- How many websites/services/operating systems have default accounts with empty passwords, allowing anyone to login?
 - Western Digital My Cloud (2019): [CVE-2019-9950](#)
 - Microsoft SQL Server (2000): [CVE-2000-1209](#)
 - D-Link (2020): [CVE-2020-8864](#)

No Password Check

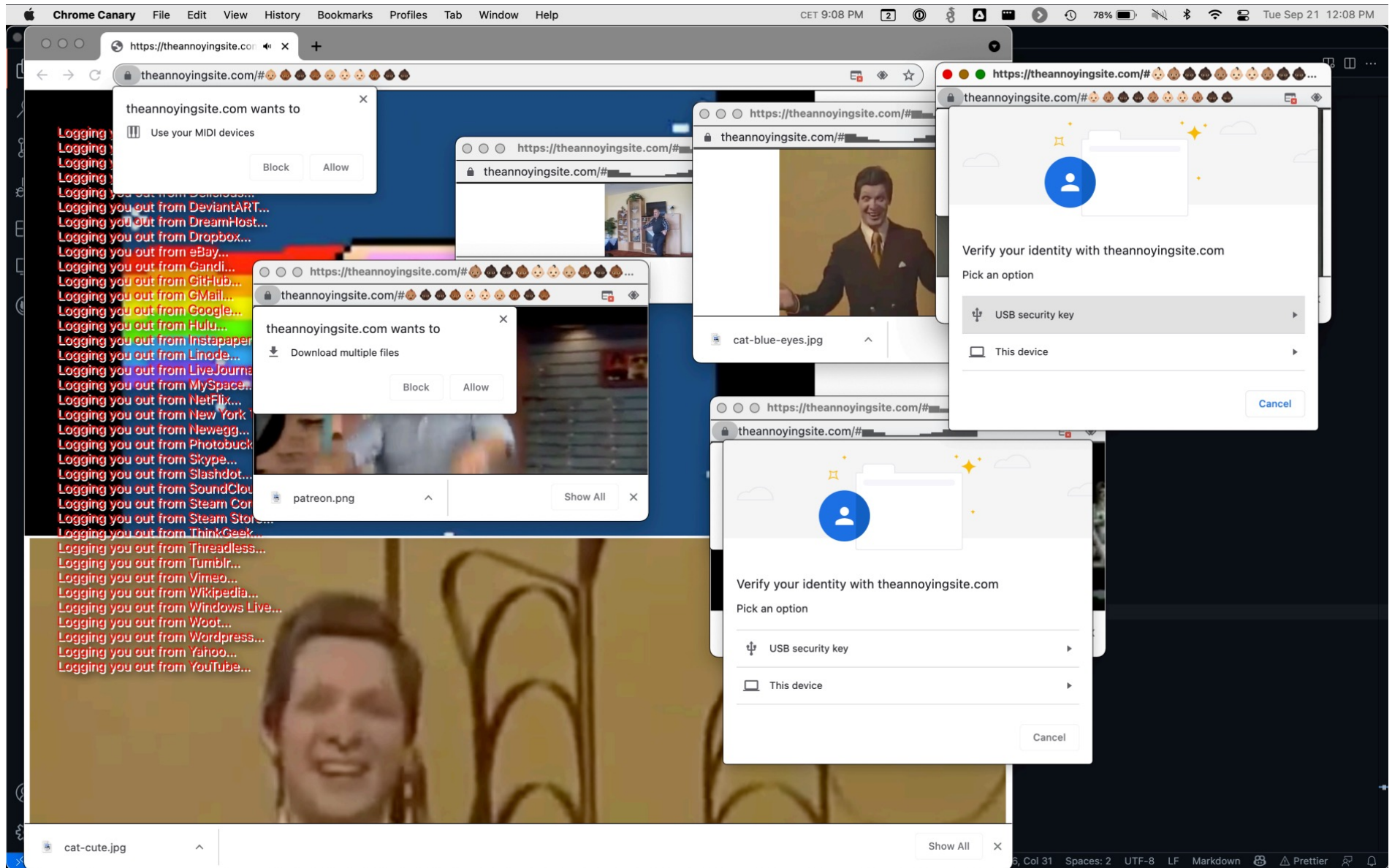
- How many websites/services/OSes have allowed login without checking the password at all?

No Password Check

- How many websites/services/operating systems have allowed login without checking the password at all?
 - IRC5 (2020): [CVE-2020-10288](#)
 - Eclipse Mosquito (2018): [CVE-2018-12551](#)
 - Keycloak (2019): [CVE-2019-14909](#)

Case Studies

- [Dropbox Security Bug Made Passwords Optional For Four Hours](#)
- [VBulletin CVE-2015-7808 in Metasploit](#)
- [The MySpace Sammy Worm](#)
- [GitHub DDoS](#)
- [The Annoying Site](#)



**You reached the end of the lecture.
Thank you and see you next week.**